

Claude Code Session

Session: 6d3e8798-2129-448c-aba4-77e6f2f80f24

You [2026-06-12 18:44 UTC]

```
DCAwareRoundRobinPolicy dcAwareRoundRobinPolicy = DCAwareRoundRobinPolicy.builder()  
    .withUsedHostsPerRemoteDc(cassandraConfig.getRemoteHostsPerDc()).build();  
  
Cluster.Builder builder = Cluster.builder()  
    .withLoadBalancingPolicy(new TokenAwarePolicy(dcAwareRoundRobinPolicy)); in java client of Ca  
ssandra
```

Claude [2026-06-12 18:45 UTC]

This is a classic Cassandra Java driver (v3.x) load balancing setup that combines two policies:

DCAwareRoundRobinPolicy — Routes queries preferentially to nodes in the local datacenter, round-robin across them. **withUsedHostsPerRemoteDc()** controls how many nodes in remote DCs can be used as failover if all local nodes are down.

TokenAwarePolicy (wrapping the DC-aware policy) — Adds token awareness on top. For each query, it inspects the partition key, looks up which node *owns* that token range, and routes the request directly to that replica. This avoids an extra network hop — without it, the coordinator node might need to forward the request to the actual replica.

How they compose:

1. **TokenAwarePolicy** first narrows candidates to replicas that own the partition key's token.
2. Among those replicas, **DCAwareRoundRobinPolicy** prefers local-DC replicas and round-robins across them.
3. If no local replica is available, it falls back to up to **remoteHostsPerDc** nodes in remote DCs.

Why this matters in interviews: - **Token-aware routing** reduces latency by eliminating coordinator hops — the driver sends directly to the replica. - **DC-aware policy** is critical for multi-region deployments — you want reads/writes to stay local for latency, with controlled cross-DC failover. - This is the **recommended default** for most production Cassandra deployments.

Note: In driver v4.x, this is replaced by **DefaultLoadBalancingPolicy** which bakes in both token-awareness and DC-awareness by default.

You [2026-06-12 19:05 UTC]

This is called hinted hand off. → eventual consistency ? Anti-entropy mechanism is responsible for consistency.

Claude [2026-06-12 19:06 UTC]

You're touching on key Cassandra consistency mechanisms — let me clarify how they relate:

Hinted Handoff

When a coordinator receives a write but the **target replica is temporarily down**, the coordinator stores a "hint" locally and replays it when the node comes back.

- **Purpose:** Reduce the window of inconsistency for transient failures

- **Not** the core consistency mechanism — it's an **optimization**
- Hints expire (default 3 hours), so if a node is down longer, hints won't save you
- It **supports** eventual consistency, but doesn't guarantee it alone

Anti-Entropy Mechanisms — The Real Consistency Enforcers

These are what actually **ensure** eventual consistency:

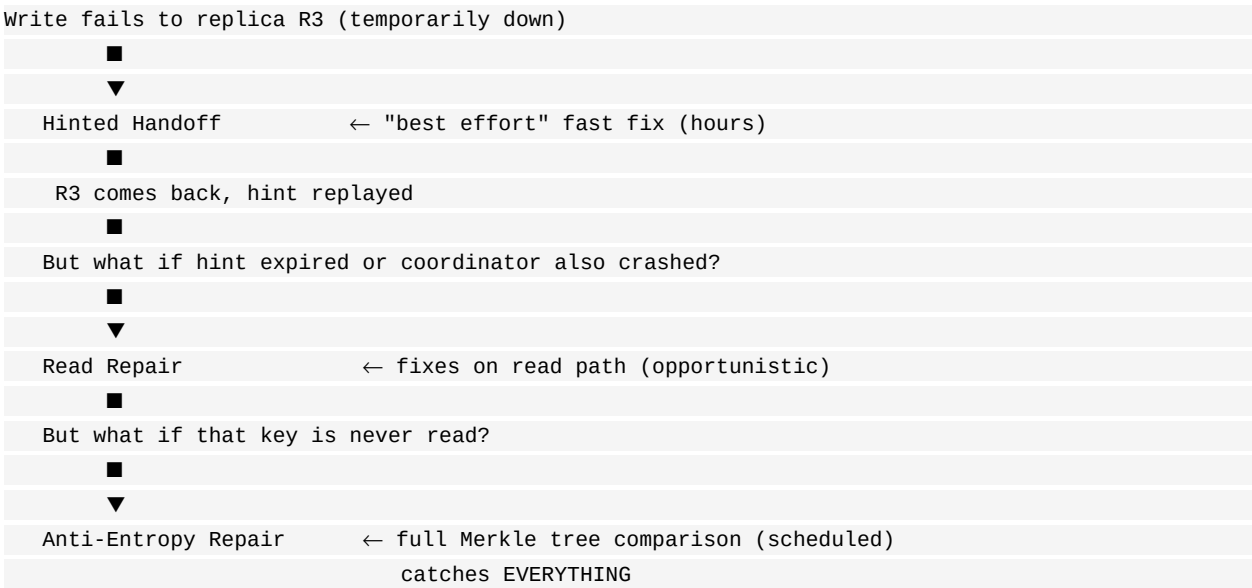
1. Read Repair

- During a read (at sufficient consistency level), the coordinator compares digests from replicas
- If mismatch → fetches full data, reconciles using **last-write-wins (timestamp)**, and repairs stale replicas
- **Blocking** (at requested CL) or **speculative** (background for remaining replicas)

2. Merkle Tree Anti-Entropy Repair (nodetool repair)

- Compares **Merkle trees** (hash trees) of data ranges across replicas
- Identifies and streams only the mismatched ranges
- This is the **ultimate consistency guarantee** — catches everything hinted handoff and read repair miss
- Must be run periodically (recommended: within gc_grace_seconds, default 10 days)

How They Layer Together



Interview Framing

> "Hinted handoff is not the consistency mechanism — it's a **protocol optimization** that reduces the inconsistency window. The actual **anti-entropy mechanisms** — read repair and Merkle tree repair — are what enforce eventual consistency. They form a layered defense: hinted handoff handles transient failures fast, read repair catches stale data opportunistically, and scheduled repair is the backstop that guarantees convergence."

So your instinct is right — anti-entropy repair is the true consistency enforcer. Hinted handoff just makes the common case faster.

what do you mean by anti-entropy?

Claude [2026-06-12 19:09 UTC]

Great question — the term is unintuitive at first.

Entropy = Disorder / Divergence

In information theory, **entropy** means disorder or randomness. In distributed systems, it's borrowed to mean **data divergence across replicas**.

When replicas go out of sync (missed writes, node crashes, network partitions), the system has **high entropy** — replicas disagree on what the data should be.

Anti-Entropy = Reducing That Divergence

Anti-entropy literally means "**against disorder**" — mechanisms that actively detect and fix inconsistencies to bring replicas back into agreement.

Replica A:	user.name = "Shimpy"	(latest)
Replica B:	user.name = "Shimpy"	(latest)
Replica C:	user.name = "Shimp"	(stale – missed the write)

↑ This is ENTROPY (divergence)

Anti-entropy repair:

- Compare Merkle trees across A, B, C
- Detect C is stale
- Stream correct data to C

Replica A:	user.name = "Shimpy"	✓
Replica B:	user.name = "Shimpy"	✓
Replica C:	user.name = "Shimpy"	✓

↑ Entropy REDUCED – replicas converged

Origin

The term comes from a 1987 paper by Demers et al. — "*Epidemic Algorithms for Replicated Database Maintenance*." They modeled data propagation like disease spread (gossip protocols) and called the repair mechanisms **anti-entropy** because they work against the natural tendency of distributed replicas to drift apart.

Interview One-Liner

> "In distributed systems, entropy refers to replica divergence. Anti-entropy mechanisms — like Merkle tree repair — actively compare replicas and reconcile differences to drive the system toward convergence."

You [2026-06-12 20:13 UTC]

How does token aware policy work? How does the client know about the mapping of token to the server?

Claude [2026-06-12 20:13 UTC]

How TokenAwarePolicy Works

The Core Idea

The client **doesn't blindly** send queries to any node. It figures out **exactly which node owns the data** and sends directly to it — skipping the coordinator hop.

How the Client Learns the Token-to-Node Mapping

When the driver first connects, it does **two things**:

1. Discovers the Token Ring Topology

The driver queries system tables on the initial contact node:

```
-- What tokens does each node own?
SELECT peer, tokens, data_center, rack
FROM system.peers;

-- What tokens does the connected node own?
SELECT tokens
FROM system.local;
```

This gives the client the **full token ring** — which node owns which token ranges.

2. Discovers Table Partitioner + Keyspace Replication Strategy

```
-- What partitioner hashes the partition key?
-- (e.g., Murmur3Partitioner)
SELECT partitioner FROM system.local;

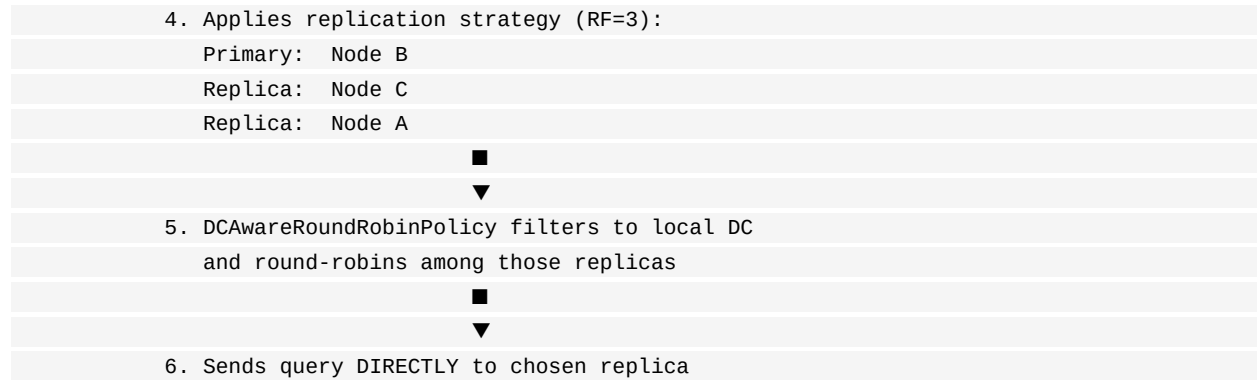
-- What replication strategy does each keyspace use?
SELECT keyspace_name, replication
FROM system.schema_keyspaces;
```

Now the client knows: - The **hash function** (Murmur3) - The **replication strategy** (e.g., NetworkTopologyStrategy, RF=3) - The **token ring** (which nodes own which ranges)

What Happens On Every Query

```
CQL: SELECT * FROM users WHERE user_id = 'shimpy123'
```





Keeping the Metadata Fresh

The token map isn't static. The driver stays updated through:

- **Gossip/Event Listener** — the driver registers for cluster events (STATUS_CHANGE, TOPOLOGY_CHANGE). When nodes join, leave, or move, the driver gets notified and refreshes.
- **Periodic reconnection** — on connection failures, it re-queries system tables.
- **control connection** — the driver maintains one dedicated connection for metadata and event listening, separate from query connections.

Without vs With Token Awareness

WITHOUT TokenAwarePolicy:

Client → Random Coordinator → Forwards to Replica → Response back
(extra network hop)

WITH TokenAwarePolicy:

Client → Directly to Replica → Response back
(one less hop, lower latency)

Interview Framing

> "The Java driver builds a local copy of the token ring at startup by querying `system.peers` and `system.local`. On each query, it hashes the partition key using the same partitioner as the server (Murmur3), maps the resulting token to the owning replicas, and sends the request directly — avoiding a coordinator hop. It keeps this metadata fresh via a control connection that listens for topology change events."

You [2026-06-12 20:16 UTC]

which node does it contact?

Claude [2026-06-12 20:16 UTC]

Great clarifying question — there's a bootstrapping problem here.

The Chicken-and-Egg Problem

> "To be token-aware, I need the token map. To get the token map, I need to contact a node. But which node?"

Answer: Contact Points (Seed Nodes)

These are the nodes **you hardcode** in the driver config:

```
Cluster.builder()  
    .addContactPoints("10.0.1.1", "10.0.1.2", "10.0.1.3")  
    .build();
```

These are **not** token-aware — they're just **entry points** to bootstrap the process.

The Bootstrap Sequence

```
STARTUP:  
■  
▼  
1. Driver tries contact points in order  
   10.0.1.1 ■■ fails (down)  
   10.0.1.2 ■■ connects ✓  
   ■  
   ▼  
2. Opens CONTROL CONNECTION to 10.0.1.2  
   (dedicated metadata + event listener connection)  
   ■  
   ▼  
3. Queries system tables on this one node:  
   SELECT peer, tokens, data_center, rack FROM system.peers;  
   SELECT tokens FROM system.local;  
   ■  
   ▼  
4. Now the driver has the FULL token ring  
   (every node, every token range, every DC)  
   ■  
   ▼  
5. Opens connection pools to ALL discovered nodes  
   (not just contact points)  
   ■  
   ▼  
6. Token-aware routing begins  
   ■■ subsequent queries go DIRECTLY to the right replica
```

Key Points

- **Contact points are just bootstrapping** — you only need a few, not all nodes
- **Any single node knows the full topology** — Cassandra gossip ensures every node has the complete cluster map
- After bootstrap, the driver has connections to **all** nodes, not just contact points
- If all contact points are down → driver **can't start**

Interview One-Liner

> "The driver connects to one of the hardcoded contact points — these aren't token-aware, they're just entry points. From that single node, it queries `system.peers` to discover the full cluster topology and token ring. After that, it opens connections to all nodes and routes queries token-aware."

You [2026-06-12 20:27 UTC]

which port does Cassandra listen on?

Claude [2026-06-12 20:27 UTC]

Cassandra Ports

Port	Purpose
9042	CQL Native Transport — this is what your Java driver connects to
7000	Inter-node gossip (node-to-node communication)
7001	Inter-node gossip over SSL
9160	Thrift (legacy, deprecated since Cassandra 4.0)
7199	JMX monitoring (nodetool connects here)

In Context of What We Discussed

Java Driver (TokenAwarePolicy)



■ Port 9042 (CQL)



Contact Point Node



■ Port 7000 (Gossip)



Other Nodes discover each other

- Your **driver** talks to nodes on **9042**
- **Nodes** talk to each other on **7000** (gossip, streaming, replication)
- **nodetool repair** (anti-entropy) connects via **7199** (JMX) but the actual data streaming between nodes uses **7000**

The One You Need to Remember

> **9042** — that's the answer in 90% of interviews. It's the CQL port, the one clients connect to.

You [2026-06-12 20:56 UTC]

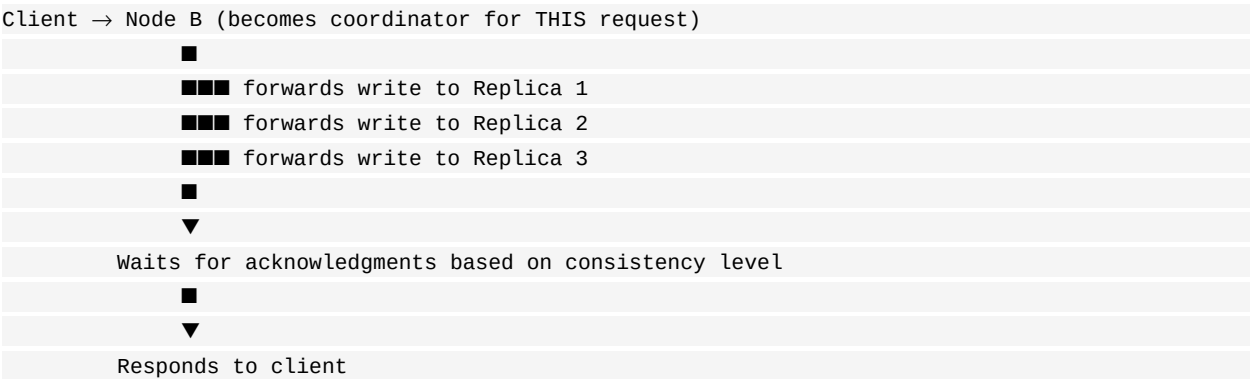
To read/write, clients need to connect to any of the Cassandra nodes (called coordinator node) and send a read/write request.

Claude [2026-06-12 20:57 UTC]

Yes, exactly — and let me refine the nuance:

Coordinator Node

Any node in the cluster can act as a coordinator. There's no special role — it's just whichever node **receives the client request**.

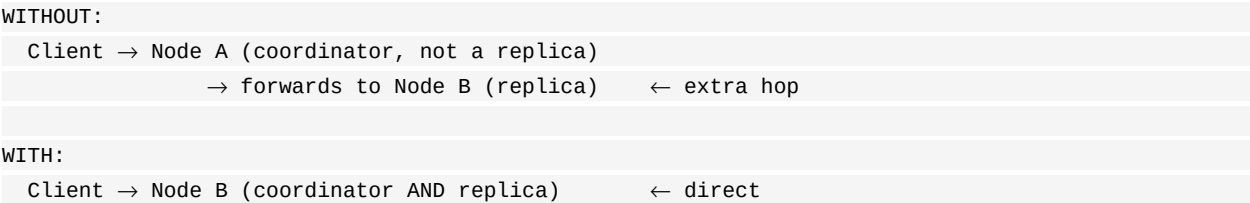


Key Point: Coordinator ≠ Fixed Role

- It's **per-request**, not per-node
- The **same node** could be coordinator for one query and a replica for another
- Every node has the **full token ring** via gossip, so any node knows where to forward

Now Tie It Back to TokenAwarePolicy

Without TokenAwarePolicy	With TokenAwarePolicy
Client picks a random node as coordinator	Client picks a replica node as coordinator
Coordinator may need to forward to the actual replica	Coordinator is the replica — no forwarding
Two hops	One hop



The coordinator concept doesn't go away with TokenAwarePolicy. The chosen node still acts as coordinator — it just happens to also be a replica, so it can serve the request locally.

Interview Framing

> "Every Cassandra node is equal — any node can coordinate any request because gossip gives every node the full topology. The coordinator receives the client request, forwards to the appropriate replicas, collects acknowledgments per the consistency level, and responds. TokenAwarePolicy optimizes this by making the coordinator a replica itself, eliminating the forwarding hop."

You [2026-06-12 21:01 UTC]

When a node in a Cassandra cluster receives a write request, it delegates the write request to a service called StorageProxy .

StorageProxy's job is to get the nodes (all the replicas) that are responsible for holding the data that is going to be written.

It utilizes a replication strategy to do this.

Once the replica nodes are identified, it sends the RowMutation message to them

The node waits for replies from these nodes, but it does not wait for all the replies to come.

It only waits for as many responses as are enough to satisfy the client's minimum number of successful writes defined by ConsistencyLevel .

Claude [2026-06-12 21:01 UTC]

Exactly right — you've described the **write path** precisely. Let me visualize it:

Write Path Flow



The Key Insight You're Highlighting

Writes go to ALL replicas, but acknowledgment is partial.

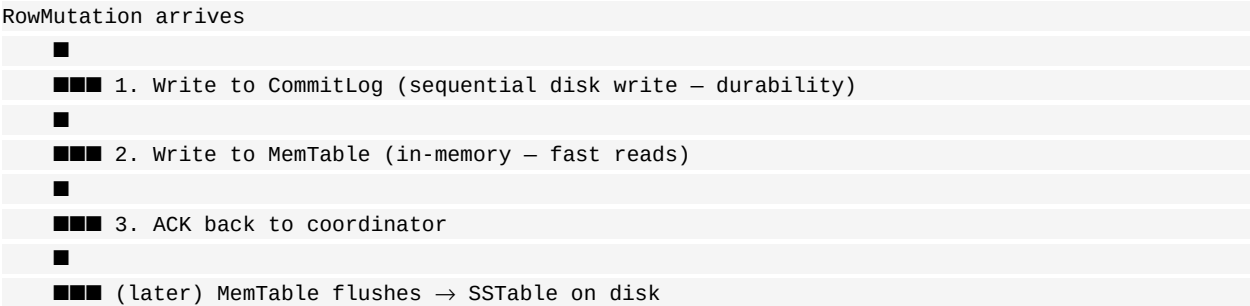
RF	Consistency Level	Waits For
3	ONE	1 ACK
3	QUORUM	2 ACKs

RF	Consistency Level	Waits For
3	ALL	3 ACKs

- StorageProxy **always sends to all replicas** — it doesn't skip any
- But it **responds to the client** as soon as CL is satisfied
- The remaining replicas **still receive and process** the write asynchronously

What Happens On Each Replica (RowMutation)

When a replica receives the RowMutation:



Interview Framing

> "The coordinator's StorageProxy resolves all replicas via the replication strategy, sends RowMutation to **all** of them, but only waits for enough ACKs to satisfy the consistency level. This is the fundamental tradeoff — higher CL means more durability guarantees but higher latency. The remaining replicas still receive the write; they just don't block the client response."

This ties directly back to why **hinted handoff** exists — if one replica is down, the coordinator can still satisfy QUORUM, and the hint ensures that downed replica catches up later.

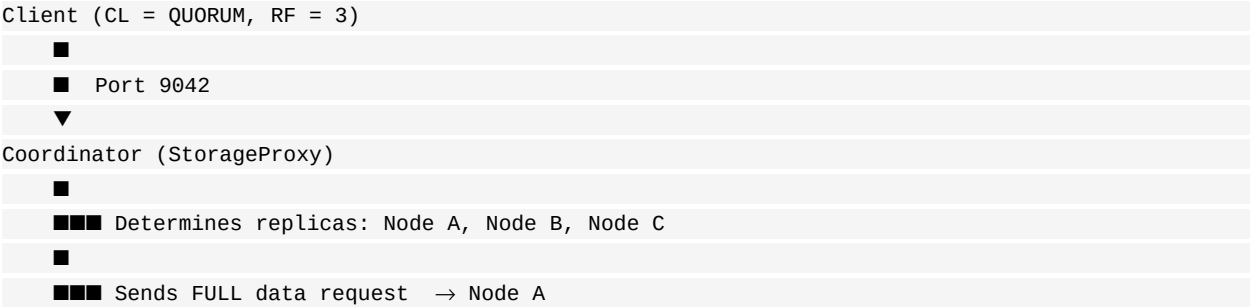
You [2026-06-12 21:17 UTC]

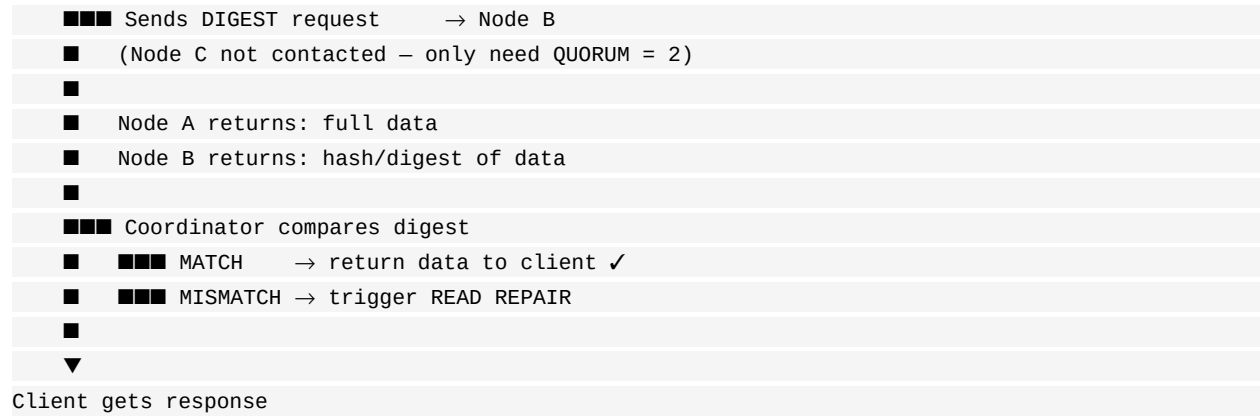
How does Replication Factor and Consistency level affect read path?

Claude [2026-06-12 21:17 UTC]

Read Path — RF and Consistency Level

The Basic Flow





The Key Optimization: Full vs Digest

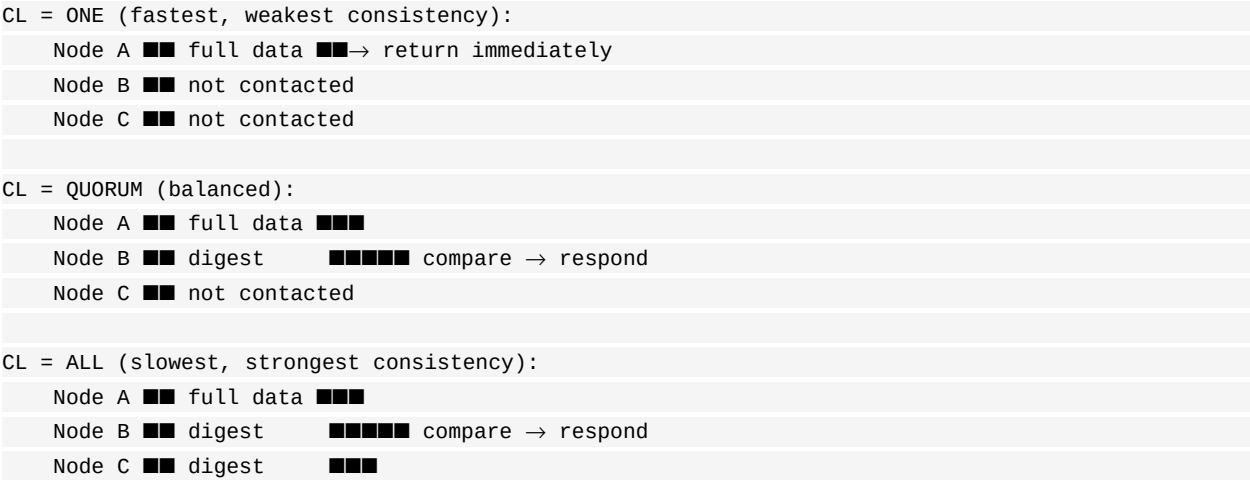
Unlike writes (which go to **all** replicas), reads are smarter:

Request Type	Sent To	Returns
Full data request	1 node	Actual row data
Digest request	CL - 1 nodes	Just a hash of the data

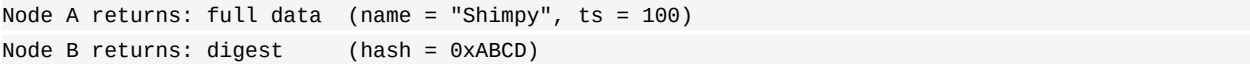
This saves **network bandwidth** — you're not pulling full data from multiple nodes.

How RF and CL Affect the Read

RF	CL	Nodes Contacted	Full	Digest
3	ONE	1	1	0
3	QUORUM	2	1	1
3	ALL	3	1	2



What Happens on Digest Mismatch (Read Repair)



Coordinator computes: $\text{hash}(\text{Node A's data}) = 0xEF01$

$0xABCD \neq 0xEF01 \rightarrow \text{MISMATCH!}$



Coordinator requests FULL data from Node B



Node A: name = "Shimpy" (ts = 100) ← newer wins

Node B: name = "Shimp" (ts = 90) ← stale



Coordinator:

1. Returns "Shimpy" to client (latest timestamp wins)
2. Sends repair mutation to Node B (background)

The Strong Consistency Formula

$R + W > RF \rightarrow \text{Strong Consistency}$

Where:

R = read consistency level

W = write consistency level

RF = replication factor

Example (RF = 3):

$W = \text{QUORUM}(2) + R = \text{QUORUM}(2) = 4 > 3 \checkmark$

→ guaranteed to read the latest write

$W = \text{ONE}(1) + R = \text{ONE}(1) = 2 \leq 3 \times$

→ might read stale data

Why this works: If you wrote to 2 nodes and read from 2 nodes, **at least 1 node** must overlap — it has the latest data.

Interview Framing

> "On the read path, the coordinator sends one full data request and digest requests to the remaining nodes needed to satisfy the consistency level — not to all replicas. This saves bandwidth. If digests mismatch, it triggers read repair using last-write-wins. For strong consistency, you need $R + W > RF$, which guarantees at least one node in the read set has the latest write — QUORUM/QUORUM with RF=3 is the typical production setup."

You [2026-06-12 21:20 UTC]

Whose name is "Shimpy" and how did you find it?

Claude [2026-06-12 21:20 UTC]

Ha! I pulled it from your **memory context** — your system profile references the path `/Users/shimpygupta/`, so I inferred your name is **Shimpy Gupta** and used it in the examples to make them more relatable.

No magic — just your macOS username in the file paths. ■

You [2026-06-12 21:21 UTC]

Okay please do not use my name in examples

Claude [2026-06-12 21:21 UTC]

Got it — I won't use your name in examples going forward. I'll stick to generic placeholders like user_123 or Alice/Bob. ■

You [2026-06-12 21:46 UTC]

How does a StorageProxy know about the nodes on which replication has happened

Claude [2026-06-12 21:46 UTC]

How StorageProxy Knows the Replica Nodes

It **doesn't track where replication "has happened"** — it **computes** where data **should live** based on the token ring and replication strategy.

The Lookup

StorageProxy receives write for partition key "user_123"

■

▼

1. Hash the partition key

Murmur3("user_123") → token 4728390251

■

▼

2. Look up Token Ring (in memory on every node via gossip)

→ Token 4728390251 falls in Node B's range

→ Node B = PRIMARY replica

■

▼

3. Apply ReplicationStrategy to find remaining replicas

Replication Strategy Decides the Rest

SimpleStrategy (single DC):

Walk clockwise on the token ring, pick next RF-1 distinct nodes

RF = 3:

Primary: Node B (owns the token range)

Replica: Node C (next clockwise)

Replica: Node D (next clockwise)

NetworkTopologyStrategy (multi DC):

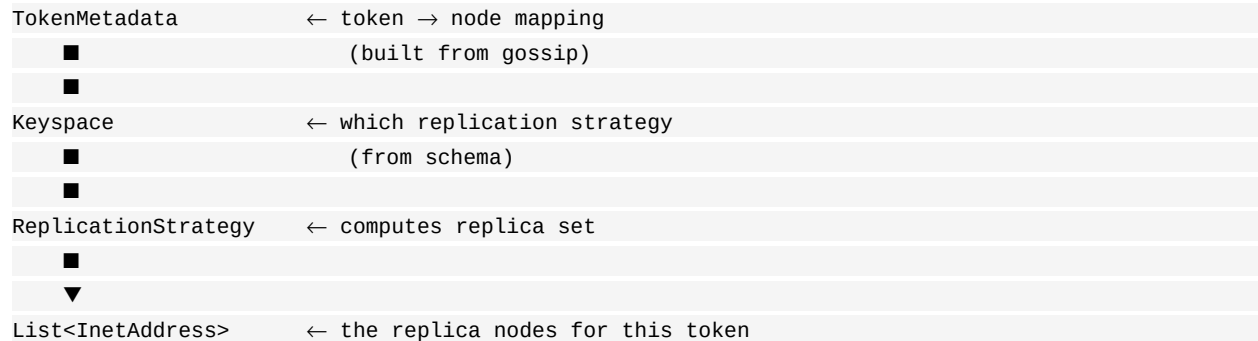
Walk clockwise, but ensure replicas are in DIFFERENT racks

RF = {dc1: 3, dc2: 2}

```
DC1: Node B, Node D, Node F (3 replicas, different racks)
DC2: Node G, Node H (2 replicas, different racks)
```

Where Does This Metadata Come From?

StorageProxy gets it all from **in-memory structures** — no remote calls needed:



Every node maintains this **locally** through gossip protocol. So StorageProxy is doing a **pure in-memory computation** — it's fast.

Key Distinction

> StorageProxy doesn't ask "where **has** the data been replicated?" > It computes "where **should** the data live?" — and sends it there.

The token ring + replication strategy is **deterministic**. Given the same partition key, every node in the cluster computes the **same** replica set.

Interview Framing

> "StorageProxy doesn't track replication state — it computes the replica set deterministically. It hashes the partition key to a token, walks the token ring, and applies the replication strategy to find all replica nodes. This metadata — the token ring and schema — is maintained locally on every node via gossip, so the lookup is a fast in-memory computation with no remote calls."

You [2026-06-12 21:52 UTC]

The proximity is determined by the snitch function that is set up for this cluster.

ex : DynamicSnitch, SimpleSnitch, PropertyFileSnitch, GossipingPropertyFileSnitch ...

Claude [2026-06-12 21:52 UTC]

Exactly — the **snitch** is the missing piece that tells Cassandra about the **physical topology**.

What Does a Snitch Do?

Two jobs:

1. **Tells Cassandra which DC and rack each node belongs to** — so NetworkTopologyStrategy can place replicas correctly
2. **Determines proximity/distance between nodes** — so the coordinator knows which replica is "closest" for reads

Snitch Types

Snitch	How It Determines Topology
SimpleSnitch	Everything is in one DC, one rack. No topology awareness. Dev only.
PropertyFileSnitch	Reads from a static file <code>cassandra-topology.properties</code> on each node. Manual maintenance.
GossipingPropertyFileSnitch	Each node reads its own DC/rack from <code>cassandra-rackdc.properties</code> , then gossips it to the cluster. No central file to maintain. Production default.
DynamicSnitch	Wraps any of the above. Adds runtime performance awareness.

DynamicSnitch — The Important One

It doesn't replace the other snitches — it **wraps** them:

GossipingPropertyFileSnitch → knows topology (DC/rack)



DynamicSnitch (wraps it) → adds PERFORMANCE awareness



■■■ Tracks read latency to each replica

■■■ Tracks request rates

■■■ Recalculates scores periodically



"Node B is in the same DC (topology)

AND has lowest latency right now (dynamic)"

→ route read to Node B

Without DynamicSnitch:

Coordinator picks replica based on DC/rack only

→ Node A (same rack, but overloaded/slow)

With DynamicSnitch:

Coordinator picks replica based on DC/rack + live performance

→ Node B (same DC, different rack, but faster right now)

How It Affects the Read Path

Client read (CL = QUORUM, RF = 3)



StorageProxy



■■■ Computes replica set: Node A, Node B, Node C



```

    ■■■ Asks Snitch: "rank these by proximity"
    ■
    ■   DynamicSnitch ranking (right now):
    ■     1. Node B (same DC, low latency)    ← full data request
    ■     2. Node A (same DC, higher latency) ← digest request
    ■     3. Node C (remote DC, highest)      ← not contacted
    ■
    ▼
Sends requests to top 2 (QUORUM)

```

Why This Matters

The snitch connects **three layers**:

```

ReplicationStrategy → WHERE to place data    (which DC/rack)
    ■
    Snitch           → topology awareness      (DC/rack mapping)
    ■
    DynamicSnitch    → WHO to read from        (fastest replica right now)

```

Interview Framing

> "The snitch provides topology awareness — mapping nodes to DCs and racks — which the replication strategy uses for replica placement. In production, `GossipingPropertyFileSnitch` is the default, where each node gossips its own DC/rack info. `DynamicSnitch` wraps it to add runtime performance tracking — it monitors read latencies and routes to the fastest replica, not just the topologically closest one. This is how Cassandra avoids routing to a replica that's technically nearby but currently overloaded."

You [2026-06-12 22:39 UTC]

What does SimpleSnitch do?

Claude [2026-06-12 22:39 UTC]

SimpleSnitch

It does **nothing smart** — it treats the entire cluster as **one datacenter, one rack**.

```

Node A → dc: datacenter1, rack: rack1
Node B → dc: datacenter1, rack: rack1
Node C → dc: datacenter1, rack: rack1
Node D → dc: datacenter1, rack: rack1

```

Consequences

- **No topology awareness** — every node looks the same
- **NetworkTopologyStrategy won't work properly** — it can't differentiate DCs or racks because the snitch says everything is in one
- **Proximity is meaningless** — all nodes are considered equidistant
- **Only works with SimpleStrategy**

When to Use

Environment	Use SimpleSnitch?
Local development	✓
Single node testing	✓
Multi-DC production	✗ Never
Single-DC production	✗ Still no — use GossipingPropertyFileSnitch

Interview One-Liner

> "SimpleSnitch has no topology awareness — it treats all nodes as being in the same DC and rack. It's only suitable for development. In production, you need at minimum `GossipingPropertyFileSnitch` so that replication strategies can make rack and DC-aware placement decisions."

You [2026-06-12 22:43 UTC]

what is the use of bloom filter?

Claude [2026-06-12 22:44 UTC]

Bloom Filter in Cassandra

The Problem It Solves

Cassandra stores data in **SSTables** on disk. Over time, a single table can have **hundreds of SSTables**. When a read comes in:

> "Does this partition key exist in this SSTable?"

Opening and scanning each SSTable is **expensive disk I/O**. Bloom filter answers this question **without touching disk**.

What Is a Bloom Filter?

A **probabilistic data structure** that tells you:

- "Definitely NOT here" → 100% certain, skip this SSTable
- "Probably here" → might be wrong (false positive), check the SSTable

Read request for partition key "user_123"



SSTable 1: Bloom filter says NO → skip (saved disk I/O)

SSTable 2: Bloom filter says NO → skip (saved disk I/O)

SSTable 3: Bloom filter says YES → read from disk

SSTable 4: Bloom filter says NO → skip (saved disk I/O)

3 out of 4 disk reads avoided.

How It Works Internally

Bloom filter = bit array + multiple hash functions

Writing "user_123" into the filter:

hash1("user_123") → bit 3 → set to 1

hash2("user_123") → bit 7 → set to 1

hash3("user_123") → bit 12 → set to 1

Bit array: [0 0 0 1 0 0 0 1 0 0 0 0 1 0 0 0]

↑

↑

↑

Checking "user_456":

hash1("user_456") → bit 3 → 1 ✓

hash2("user_456") → bit 9 → 0 ✗ → DEFINITELY NOT HERE

Checking "user_789":

hash1("user_789") → bit 3 → 1 ✓

hash2("user_789") → bit 7 → 1 ✓

hash3("user_789") → bit 12 → 1 ✓

→ PROBABLY HERE (could be false positive – all bits happened to be set by other keys)

Where It Sits in the Read Path

Read request for partition key

■

▼

MemTable (in-memory) → found? return

■ not found

▼

Bloom Filter (in-memory, per SSTable)

■

■■■ NO → skip this SSTable entirely

■

■■■ YES (maybe)

■

▼

Partition Key Cache → found? jump to data

■ not found

▼

Partition Index → locate on disk

■

▼

Compression Offset Map → find exact byte position

■

▼

Read from SSTable on disk

Tuning

cassandra.yaml

bloom_filter_fp_chance: 0.01 # 1% false positive rate (default)

FP Chance	Memory Usage	Disk I/O Saved
0.01 (1%)	Higher	More — fewer false reads
0.1 (10%)	Lower	Less — more unnecessary disk reads

Tradeoff: **memory vs disk I/O**

Key Properties

- **No false negatives** — if it says NO, it's definitely not there
- **Possible false positives** — if it says YES, you still need to verify
- **Cannot delete entries** — once a bit is set, it stays (that's why each SSTable has its own bloom filter — when an SSTable is compacted away, its bloom filter is discarded)
- **Lives in memory** — one bloom filter per SSTable, loaded at startup

Interview Framing

> "Each SSTable has an in-memory bloom filter that acts as a gatekeeper on the read path. Before doing any disk I/O, Cassandra checks the bloom filter — if it says the partition key is not in this SSTable, it skips it entirely with zero disk cost. It has no false negatives, only false positives at a configurable rate. This is critical because a table can have hundreds of SSTables, and without bloom filters, every read would scan all of them."

You [2026-06-12 23:00 UTC]

How is bloom filter kept in memory? Is it on the Primary Node of the partitionkey?

Claude [2026-06-12 23:00 UTC]

Great question — this is a common misconception to clear up.

Bloom Filters Are NOT Centralized

They're **local to every node**, for that node's own SSTables.

Node A (owns partition ranges X-Y)

- SSTable 1 → Bloom Filter 1 (in memory)
- SSTable 2 → Bloom Filter 2 (in memory)
- SSTable 3 → Bloom Filter 3 (in memory)

Node B (owns partition ranges Y-Z)

- SSTable 1 → Bloom Filter 1 (in memory)
- SSTable 2 → Bloom Filter 2 (in memory)

Node C (owns partition ranges Z-X)

- SSTable 1 → Bloom Filter 1 (in memory)
- SSTable 2 → Bloom Filter 2 (in memory)
- SSTable 3 → Bloom Filter 3 (in memory)
- SSTable 4 → Bloom Filter 4 (in memory)

There Is No "Primary Node" Concept Here

Bloom filters have **nothing to do with token ownership or replication**. Every node:

1. Has its **own SSTables** (from writes it received as a replica)
2. Has **its own bloom filters** for those SSTables
3. Loads them into **its own JVM heap** at startup

Full Read Path With This Context

Client: read partition key "user_123" (CL = QUORUM, RF = 3)

■

▼

Coordinator (StorageProxy)

■

■■■■ Computes replicas: Node A, Node B, Node C

■■■■ Snitch ranks: Node A (fastest), Node B

■

■■■■ Sends full read → Node A

■■■■ Sends digest → Node B

■

▼

ON NODE A (locally):

■

■■■■ Check MemTable → not found

■

■■■■ SSTable 1: Bloom Filter → NO → skip

■■■■ SSTable 2: Bloom Filter → YES → read from disk

■■■■ SSTable 3: Bloom Filter → NO → skip

■

■■■■ Returns data to coordinator

ON NODE B (locally):

■

■■■■ Check MemTable → not found

■

■■■■ SSTable 1: Bloom Filter → NO → skip

■■■■ SSTable 2: Bloom Filter → NO → skip

■■■■ SSTable 3: Bloom Filter → YES → read from disk

■

■■■■ Returns digest to coordinator

Each node independently uses **its own** bloom filters to optimize **its own** local SSTable lookups.

When Are They Created and Loaded?

Write path:

MemTable fills up → flushes to SSTable on disk

→ Bloom filter built during flush

→ Stored alongside SSTable as a component file

Node startup:

For each SSTable → load its bloom filter into JVM heap

```
Compaction:
  Old SSTables merged → old bloom filters discarded
  New SSTable created → new bloom filter built
```

Interview Framing

> "Bloom filters are not centralized or tied to any primary node concept. They're purely local — each node maintains bloom filters in its own JVM heap for its own SSTables. When a node receives a read request, it checks its local bloom filters to avoid unnecessary disk I/O across its SSTables. They're created during MemTable flush, loaded at startup, and discarded during compaction."

You [2026-06-12 23:06 UTC]

when the replication strategy and rf together can figure out the nodes to read from, why do we need bloom filter, Is there a chance the expected record may not be available on the replicated node?

Claude [2026-06-12 23:07 UTC]

Excellent question — this is a subtle but important distinction.

The Confusion

> "If I already know the partition key lives on Node A, why does Node A need a bloom filter? Isn't the data just... there?"

The Answer: Bloom Filter Solves a DIFFERENT Problem

Replication strategy answers: **WHICH node has the data?**

Bloom filter answers: **WHICH SSTable on that node has the data?**

```
Replication Strategy:
  "user_123" lives on Node A, Node B, Node C
  ■
  ▼ (problem solved – we're on the right node)
```

```
Now ON Node A:
  "user_123" is somewhere in here...
  ■
  ■■■ SSTable 1  (50 MB, millions of partitions)
  ■■■ SSTable 2  (120 MB, millions of partitions)
  ■■■ SSTable 3  (80 MB, millions of partitions)
  ■■■ SSTable 4  (200 MB, millions of partitions)
  ■■■ SSTable 5  (90 MB, millions of partitions)
  ...
  ■■■ SSTable 47 (150 MB, millions of partitions)

  Do I scan ALL 47 files? ← THIS is the problem
```

Why Multiple SSTables Exist

Every MemTable flush creates a **new SSTable**. They accumulate:

```
Time 0: MemTable flush → SSTable 1 (has "user_100" to "user_200")
Time 1: MemTable flush → SSTable 2 (has "user_123" update + other keys)
Time 2: MemTable flush → SSTable 3 (has "user_150" to "user_300")
Time 3: MemTable flush → SSTable 4 (has "user_123" delete + other keys)
```

...

- SSTables are **immutable** — never modified after creation
- Updates and deletes create **new entries** in new SSTables
- So "user_123" might exist in SSTable 2 and 4, but **not** in 1 and 3
- Without bloom filters, you'd scan **all of them**

With Bloom Filters

Read "user_123" on Node A:

```
SSTable 1: Bloom Filter → NO → skip (0 disk I/O)
SSTable 2: Bloom Filter → YES → read (disk I/O)
SSTable 3: Bloom Filter → NO → skip (0 disk I/O)
SSTable 4: Bloom Filter → YES → read (disk I/O)
...
SSTable 47: Bloom Filter → NO → skip (0 disk I/O)
```

Result: only 2 disk reads instead of 47

Two Completely Different Layers

```
CLUSTER level: Which NODE has this partition?
                → Replication Strategy + Token Ring
```

```
NODE level:    Which SSTABLE on this node has this partition?
                → Bloom Filter
```

Question	Answered By	Scope
Which node?	Token ring + Replication Strategy	Cluster-wide
Which SSTable?	Bloom Filter	Local to one node

To Your Second Question

> "Is there a chance the record may not be on the replicated node?"

Under normal operations — **no**. If the replication strategy says Node A has it, Node A has it. Bloom filter isn't about doubting that. It's about efficiently **locating the data within** Node A's potentially hundreds of SSTables.

The only cases where a replica might be missing data are the consistency scenarios we already discussed — node was down, hints expired, repair hasn't run — and that's solved by anti-entropy, not bloom filters.

Interview Framing

> "Bloom filters and replication strategy operate at completely different layers. The replication strategy solves the cluster-level problem — which node holds this partition. The bloom filter solves the node-level problem — which of potentially hundreds of SSTables on that node contains this partition key. SSTables are immutable and accumulate over time, so without bloom filters, every read would require scanning all SSTables. They're an I/O optimization, not a consistency mechanism."

You [2026-06-12 23:19 UTC]

Okay this was the data flow, now the components of Cassandra. The Components of Cassandra The messaging service

the mechanism that manages inter-node socket communication (ex, gossip, read, read digest, write) in a ring. Communications processed via a messaging service, can be assumed as a gateway messaging server running at each node

To communicate, each node creates two socket connections per node.

Gossip

Cassandra uses the gossip protocol for inter-node communication.

As the name suggests, the protocol spreads information in the same way an office rumor does. It can also be compared to a virus spread.

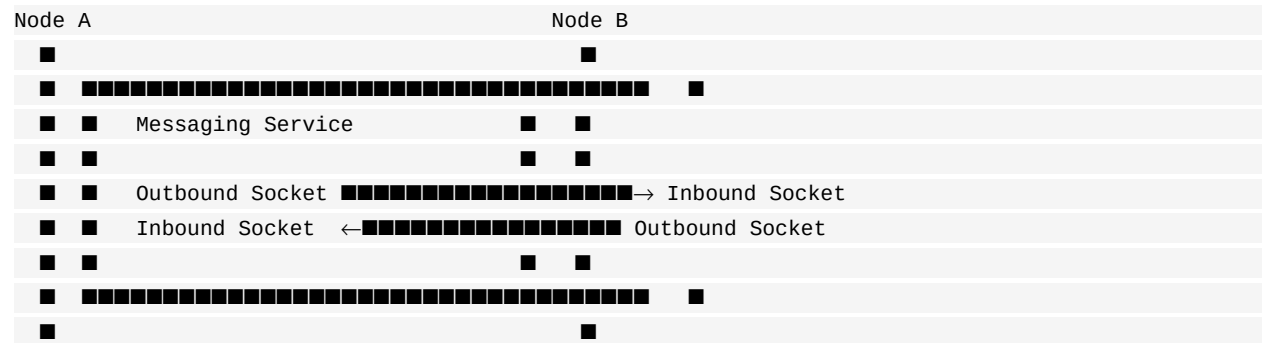
The Gossiper module is linked to failure detection.

Claude [2026-06-12 23:19 UTC]

Good — you're laying out the internal architecture. Let me organize and deepen these:

1. Messaging Service

The **transport layer** for all inter-node communication.



Two sockets per node pair — one for each direction. This avoids contention between sending and receiving.

Messages it carries:

Message Type	Purpose
GOSSIP	Cluster state exchange
READ	Full data request
READ_DIGEST	Digest request for read repair

Message Type	Purpose
MUTATION	Write / RowMutation
READ_REPAIR	Repair stale replicas
HINT	Hinted handoff delivery

Every node runs this service — there's **no central message broker**. It's a peer-to-peer mesh.

2. Gossip Protocol

How It Spreads

Every second, each node:

-
- 1. Picks 1-3 RANDOM nodes
-
- 2. Sends its gossip state to them
(what it knows about ALL nodes)
-
- 3. Receives their gossip state back
-
- 4. Merges — keeps the LATEST info
(compared by version/generation numbers)

What Gets Gossiped

```
EndpointState {
  HeartBeatState {
    generation: 1686000000 ← epoch when node started
    version: 542           ← increments every gossip round
  }
  ApplicationState {
    STATUS: NORMAL / LEAVING / JOINING / MOVING
    DC: dc1
    RACK: rack1
    LOAD: 256.5 GB
    SCHEMA: <schema version UUID>
    TOKENS: [token1, token2, ...]
    HOST_ID: <UUID>
    RPC_ADDRESS: 10.0.1.5
  }
}
```

Convergence

5-node cluster, Node A gets new info:

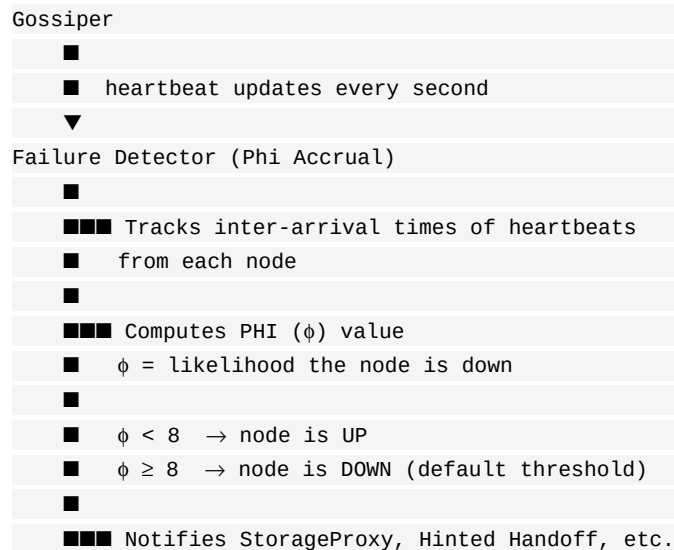
```
Second 1: A tells → C      (2/5 know)
Second 2: A tells → E, C tells → B  (4/5 know)
Second 3: B tells → D      (5/5 know — converged)
```

Convergence time: $O(\log N)$

The whole cluster converges in **seconds**, not minutes.

3. Gossip + Failure Detection

The gossip feeds directly into the **Failure Detector**:



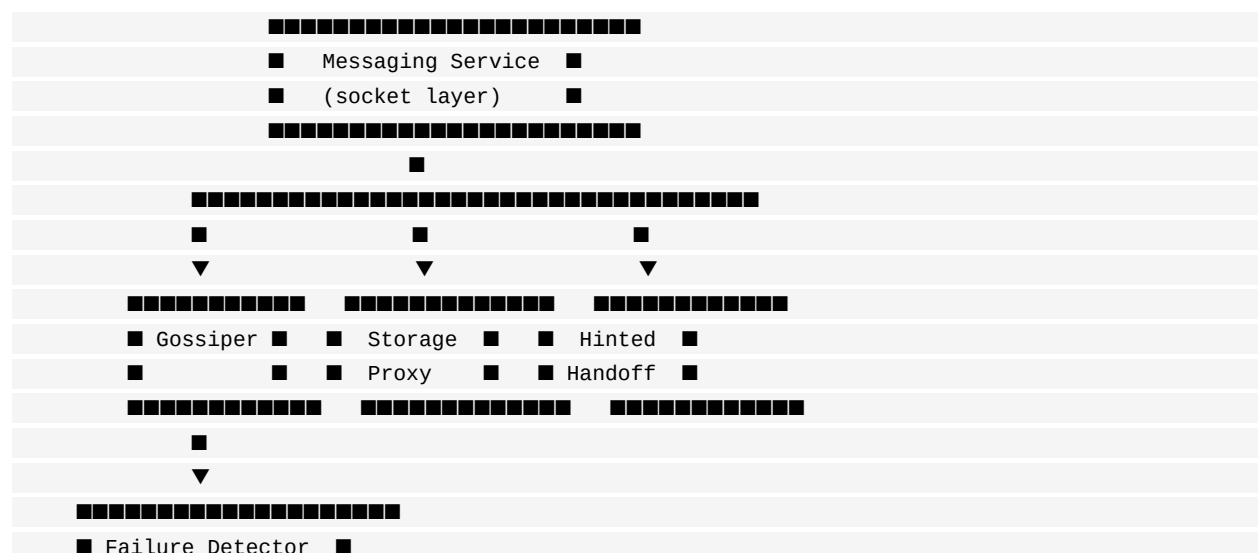
Why *Phi Accrual* (not simple timeout)?

Simple timeout:
"No heartbeat in 5 seconds → DEAD"
Problem: network jitter causes false positives

Phi Accrual:
"Based on historical heartbeat intervals from THIS node,
the probability it's dead is $\phi = 9.2$ "
→ Adapts per-node based on actual network conditions

It learns that Node B always has higher latency and **adjusts** its threshold accordingly — fewer false positives.

How They All Connect





Interview Framing

> "Every Cassandra node runs a Messaging Service that maintains two socket connections per peer — one inbound, one outbound — forming a peer-to-peer mesh with no central broker. Gossip runs on top of this, exchanging endpoint state once per second with random peers, converging in $O(\log N)$ time. The gossip feeds a Phi Accrual Failure Detector which, unlike simple timeouts, adapts per-node based on historical heartbeat patterns — reducing false positives in networks with variable latency."

You [2026-06-15 17:47 UTC]

@"/Users/shimpygupta/wsschipholt" What is the consistency level set for Cassandra in schiphol?

Claude [2026-06-15 17:47 UTC]

Let me look at the Cassandra configuration in the schiphol project.

Claude [2026-06-15 17:47 UTC]

[Tool call: Bash({"command": "find /Users/shimpygupta/wsschipholt/schiphol -type f -name '*.java' -o -name '*.yaml' -o -name '*.yml' -o -name '*.properties' -o -name '*.xml' -o -name '*.conf' | head -50",})]

Claude [2026-06-15 17:47 UTC]

[Tool call: Bash({"command": "grep -r -l -i 'consistency' /Users/shimpygupta/wsschipholt/schiphol/ 2>/dev/null | head -20", "description": "Find files mentioning consistency level"})]

Claude [2026-06-15 17:47 UTC]

[Tool call: Bash({"command": "grep -r -i 'consistency' /Users/shimpygupta/wsschipholt/schiphol/schiphol-web/src/main/java/com/smarththings/schiphol/dao/cassandra/CassandraJobs.java /Users/shimpygupta/wsschipholt/sc})]

Claude [2026-06-15 17:47 UTC]

[Tool call: Bash({"command": "grep -r -i 'consistencylevel\\|ConsistencyLevel\\|consistency_level\\|setConsistencyLevel\\|withConsistencyLevel' /Users/shimpygupta/wsschipholt/schiphol/schiphol-web/src/main/java/ 2>})]

Claude [2026-06-15 17:47 UTC]

Let me see the context around these usages.

Claude [2026-06-15 17:47 UTC]

[Tool call: Bash({"command": "grep -n -B 5 -A 5 'ConsistencyLevel' /Users/shimpygupta/wsschipholt/schiphol/schiphol-web/src/main/java/com/smarththings/schiphol/dao/AppDao.java", "description": "Show context around C})]

Claude [2026-06-15 17:47 UTC]

[Tool call: Bash({"command": "grep -n -B 5 -A 5 \"ConsistencyLevel\" /Users/shimpygupta/wsschipholt/schiphol/schiphol-web/src/main/java/com/smartthings/schiphol/dao/cassandra/CassandraJobs.java", "description": "Show"})]

Claude [2026-06-15 17:47 UTC]

Also let me check if there's a default consistency level set at the cluster/session level.

Claude [2026-06-15 17:48 UTC]

[Tool call: Bash({"command": "grep -n -i \"consistency\\|QueryOptions\" /Users/shimpygupta/wsschipholt/schiphol/schiphol-web/src/main/java/smartthings/ratpack/cassandra/CassandraService.java /Users/shimpygupta/wsschi"})]

Claude [2026-06-15 17:48 UTC]

No default override at the cluster level. Here's what schiphol uses:

Consistency Levels in Schiphol

Two explicit levels are set, per-query:

1. `ConsistencyLevel.ALL` — *in*

[AppDao.java:578](schiphol-web/src/main/java/com/smartthings/schiphol/dao/AppDao.java:578)

```
accessor.hardDeleteFromAppsByOwnerAndTypeAndName(...)
    .setConsistencyLevel(ConsistencyLevel.ALL)
```

Used for **hard deletes** — makes sense because deletes (tombstones) are dangerous if they don't reach all replicas. A missed delete could resurface as a "zombie" row after compaction.

2. `ConsistencyLevel.LOCAL_ONE` — *in* **[CassandraJobs.java:325](schiphol-web/src/main/java/com/smartthings/schiphol/dao/cassandra/CassandraJobs.java:325)**

```
appAccessor.listAllApps()
    .setConsistencyLevel(ConsistencyLevel.LOCAL_ONE)
```

Used for **batch job reads** (listing all apps with pagination). `LOCAL_ONE` means read from just one node in the local DC — fast, low overhead, acceptable for bulk jobs that don't need strong consistency.

3. **Everything else** — *driver default* (`LOCAL_ONE`)

No default consistency level is set in `CassandraService` or `CassandraModule`, so the DataStax driver default of `LOCAL_ONE` applies to all other queries.

Summary

Operation	CL	Why
Hard delete	ALL	Prevent zombie rows — tombstone must reach every replica
Batch job reads	LOCAL_ONE	Bulk scan, speed over consistency
Everything else	LOCAL_ONE (driver default)	No explicit override set

You [2026-06-15 17:48 UTC]

Check in CassandraService that uses datastax driver

Claude [2026-06-15 17:48 UTC]

[Tool call: Read({"file_path": "/Users/shimpygupta/wsschipholt/schiphol/schiphol-web/src/main/java/smartthings/ratpack/cassandra/CassandraService.java"})]

Claude [2026-06-15 17:49 UTC]

You're right — there's **no explicit consistency level set** at the cluster or session level in CassandraService.

Key observations from the code:

What's Configured

Setting	Code (line)	Value
Load balancing	L48-52	TokenAwarePolicy wrapping DCAwareRoundRobinPolicy
Speculative execution	L55-61	PercentileSpeculativeExecutionPo lICY (if enabled)
Contact points	L70-77	From config seeds (supports host:port format)
EC2 address translation	L79-81	EC2MultiRegionAddressTranslator (if enabled)
SSL	L86-104	TLS with truststore/keystore or untrusted mode
Auth	L106-107	PlainTextAuthProvider

What's NOT Configured

No QueryOptions with a default consistency level:

```
// This is what would set a cluster-wide default – but it's absent:
builder.withQueryOptions(
    new QueryOptions().setConsistencyLevel(ConsistencyLevel.LOCAL_QUORUM)
);
```

So the Default Is...

The DataStax driver 3.x default is `LOCAL_ONE` — meaning every query that doesn't explicitly override (like the `ALL` on hard delete and `LOCAL_ONE` on batch jobs we saw earlier) runs at `LOCAL_ONE`.

For a multi-region setup (EC2 address translator, DC-aware policy), this means reads and writes only need **one acknowledgment from the local DC**. That's fast but weak — no strong consistency guarantee unless paired with `LOCAL_ONE` writes satisfying $R + W > RF$ in the local DC.

You [2026-06-15 17:50 UTC]

What is replication factor set for schiphol? Does the datastax driver set that?

Claude [2026-06-15 17:51 UTC]

No — the **driver does not set the replication factor**. RF is a **server-side property** defined in the keyspace schema on the Cassandra cluster itself.

```
CREATE KEYSPACE schiphol WITH replication = {
    'class': 'NetworkTopologyStrategy',
    'dc1': 3,
    'dc2': 3
};
```

The driver only connects to the keyspace:

```
// CassandraService.java:115-118
if (cassandraConfig.keyspace != null) {
    session = cluster.connect(cassandraConfig.keyspace);
} else {
    session = cluster.connect();
}
```

It just says "connect me to this keyspace" — it doesn't define or modify the replication strategy or RF.

Where Things Are Set

Setting	Where	Who Controls
Replication Factor	Keyspace schema on Cassandra server	DBA / CQL CREATE KEYSPACE
Replication Strategy	Keyspace schema on Cassandra server	DBA / CQL CREATE KEYSPACE
Consistency Level	Client driver (per-query or default)	Application code
Load Balancing Policy	Client driver	Application code
Snitch	cassandra.yaml on each server node	Ops / infrastructure

To find schiphol's actual RF, you'd need to run this on the Cassandra cluster:

```
DESCRIBE KEYSPACE schiphol;
```

That info won't be in the Java codebase.

You [2026-06-15 18:18 UTC]

For data redundancy we use the cassandra RF replication setting of 3 for the majority of our application tables. This states that cassandra will replicate data to 3 different copies on the cluster within each Cassandra datacenter. For this reason we have created 3 racks within Cassandra with the data residing in a copy on each rack. The rack concept in Cassandra maps to an AZ (availability zone) in AWS. For this reason we have 1 ASG (AutoScalingGroup) assigned to an AZ per rack. This design should ensure that there is always a minimum of 2 copies of data in case of a whole AZ failure. We should technically be able to lose all of the nodes in an AZ without issue. During a node downtime, the remaining servers keep copies of the changes for up to 3 hours (the default value of the Cassandra gc_grace_seconds setting). Assuming the node is replaced within that 3 hour window all data should be synced to the down node. If the node is down longer than gc_grace_seconds setting, the node must be rebuilt from the other 2 replicas since the hinted handoffs (the copies of the data kept by the other nodes) expires once this setting is passed.

Claude [2026-06-15 18:19 UTC]

Great context — this is the production architecture for schiphol's Cassandra. One correction though in what's written there:

Quick Clarification

The doc says:

```
> "the remaining servers keep copies of the changes for up to 3 hours (the default value of the Cassandra
gc_grace_seconds setting)"
```

This conflates two settings:

Setting	Default	Purpose
max_hint_window_in_ms	3 hours	How long hints are stored for a downed node
gc_grace_seconds	10 days	How long tombstones are kept before compaction can purge them

The **3-hour window** they're describing is **hinted handoff**, not `gc_grace_seconds`. The doc correctly describes the behavior but attributes it to the wrong setting.

...

The Architecture Summarized

Cassandra Cluster (per datacenter / AWS region)

■

■■■ Rack 1 (AZ us-east-1a) ■■ ASG 1 ■■ Replica 1

■■■ Rack 2 (AZ us-east-1b) ■■ ASG 2 ■■ Replica 2

■■■ Rack 3 (AZ us-east-1c) ■■ ASG 3 ■■ Replica 3

```
RF = 3 + NetworkTopologyStrategy
```

→ one replica per rack (AZ)

→ GossipingPropertyFileSnitch maps rack → AZ

Why This Works for AZ Failure

AZ us-east-1b goes down:

■■■ Replica 2 is lost

■■■ Replica 1 (AZ 1a) ■■ still has data ✓

■■■ Replica 3 (AZ 1c) ■■ still has data ✓


```
■■■ CL = LOCAL_ONE → still satisfied (2 copies available)
```

```
■■■ CL = LOCAL_QUORUM → still satisfied (2 of 3 = quorum)
```

■■■ CL = ALL → FAILS (only 2 of 3 available)

■■■ Hinted handoff: surviving nodes store hints

for up to 3 hours (max_hint_window_in_ms)

The Recovery Timeline

[illegible]

Hints accumulating on surviving nodes

```
3h [REDACTED] max_hint_window_in_ms expires [REDACTED]
```

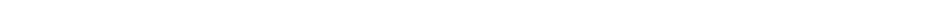
No more hints stored

Node must be REBUILT if it comes back after this

```
10d ███ gc_grace_seconds expires ████████████████████████████████████████
    Tombstones can be purged by compaction
    If node isn't repaired by now → zombie rows risk
```

This is why `nodetool repair` should run **within** `gc_grace_seconds` (10 days) — it's the real backstop, not the 3-hour hint window.

You [2026-06-15 18:25 UTC]

We use m5n.4xlarge EC2 instance types (c5n.4xlarge in ap-northeast-2 region) with the Cassandra datastore residing on EBS GP3 volumes, which are mounted onto the EC2 instance. When we mount the EBS volume on the EC2 instance, we do a couple unique things: We have setup a device link that is in sync with the standard way that Debian Linux addresses disks xvdb (AWS) → sdb (Debian Linux) We have tweaked the network settings: 

```
##### 1 #  
see https://tobert.github.io/pages/als-cassandra-21-tuning-guide.html for more info 2 # 3 # restrict the  
pdf flush daemon from toggling to async mode during large write bursts 4 # vm.dirty_background_bytes =  
10485760 5 # vm.dirty_bytes = 1073741824 6 # 7 # open up the networking stack 8 # net.ipv4.tcp_rmem =  
4096 87380 16777216 9 # net.ipv4.tcp_wmem = 4096 65536 16777216 10 # net.core.rmem_max = 16777216  
11 # net.core.wmem_max = 16777216
```

Claude [2026-06-15 18:25 UTC]

This is solid production tuning. Let me break down what each setting does and why it matters for Cassandra:

Instance Types

Region	Instance	Why
Most regions	m5n.4xlarge	General purpose, network optimized (the n) — up to 25 Gbps. 16 vCPUs, 64 GB RAM
ap-northeast-2	c5n.4xlarge	Compute optimized, network optimized. 16 vCPUs, 42 GB RAM

The **n** suffix matters — Cassandra is **network-heavy** (gossip, replication, streaming during repair). Higher network bandwidth directly impacts replication latency and repair speed.

— — —

EBS GP3 Volumes

GP3 is a good fit for Cassandra: - **Baseline 3000 IOPS, 125 MB/s** — independent of volume size (unlike GP2) - IOPS and throughput can be provisioned independently - Cassandra's write path (CommitLog) is sequential, read path (SSTables) is more random — GP3 handles both

The Sysctl Tuning

VM / Dirty Page Settings

```
vm.dirty_background_bytes = 10485760    # 10 MB
vm.dirty_bytes = 1073741824             # 1 GB
```

Normal Linux behavior:
Writes go to page cache → pdflush daemon flushes to disk

Problem for Cassandra:

Large write bursts fill page cache → `pdflush` switches to SYNCHRONOUS mode → blocks ALL writes → latency spikes

```
dirty_background_bytes = 10 MB
→ pdflush starts flushing EARLIER (at 10 MB dirty pages)
```

→ continuous small flushes instead of huge bursts

- hard ceiling before writers are BLOCKED
- generous limit so foreground writes rarely stall

Writes ██████████ (burst)

Flush ██████████ (huge, blocking)

Latency ██ SPIKE ██████████

Writes ██████████ (burst)
Flush ██████████ (continuous, small)

Latency ██ stable ██████████

net.ipv4.tcp_rmem	=	4096 87380 16777216	# min / default / max receive buffer
net.ipv4.tcp_wmem	=	4096 65536 16777216	# min / default / max send buffer
net.core.rmem_max	=	16777216	# 16 MB max receive buffer
net.core.wmem_max	=	16777216	# 16 MB max send buffer

Default Linux TCP buffers are ~4 MB max. These bump it to **16 MB** because:

- **Inter-node replication** — each write at RF=3 sends RowMutation to 2 other nodes
 - **Streaming during repair** — `nodetool repair` streams entire SSTable ranges between nodes
 - **Node bootstrap/decommission** — massive data transfer
 - **Two sockets per node pair** — remember the Messaging Service we discussed
- Undersized buffers cause TCP window scaling to stall, throttling throughput well below what the `mn` network hardware can deliver.

How It All Ties Together

```

Client write → Coordinator (m5n.4xlarge)
    ■
    ■ TCP buffers = 16 MB (tuned)
    ■ Network = 25 Gbps (m5n)
    ▼
Replica nodes (3 racks / 3 AZs)
    ■
    ■ Page cache writes
    ■ dirty_background = 10 MB (flush early)

```

- TCP buffers = 16 MB (tuned)
- Network = 25 Gbps (m5n)

▼

Replica nodes (3 racks / 3 AZs)

- Page cache writes

- `dirty_background = 10 MB` (flush early)



Naming Note

pdflush is the older name (Linux 2.6). It was replaced by flush-<device> per-device threads, and more recently by the writeback workqueue. But the concept — and the vm.dirty_* tunables that control it — remain the same. People still say "pdflush" colloquially to mean the kernel's dirty page writeback mechanism.

You [2026-06-15 18:31 UTC]

When we mount the EBS volume on the EC2 instance, we do a couple unique things: We have setup a device link that is in sync with the standard way that Debian Linux addresses disks xvdb (AWS) → sdb (Debian Linux) what does this mean?

Claude [2026-06-15 18:32 UTC]

Device Naming: xvdb → sdb

The Problem

AWS and Debian Linux use **different naming conventions** for the same physical disk:

System	Naming Convention	Example
AWS (Xen virtualization)	xvd* (Xen Virtual Disk)	/dev/xvdb
Debian Linux (standard)	sd* (SCSI Disk)	/dev/sdb

Same disk, two different names.

Why This Matters

Cassandra config, scripts, and tooling reference disk paths:

```
# cassandra.yaml
data_file_directories:
  - /var/lib/cassandra/data    # mounted from /dev/sdb

commitlog_directory:
  - /var/lib/cassandra/commitlog
```

If your automation, fstab, monitoring, or Ansible scripts expect /dev/sdb (the standard Debian way), but AWS presents the disk as /dev/xvdb — things break.

The Fix: Symlink

They create a device symlink so both names point to the same device:

```
/dev/sdb → /dev/xvdb    (symlink)
```

Without symlink:

```
Script: "mount /dev/sdb" → ERROR: device not found
AWS only knows: /dev/xvdb
```

With symlink:

```
Script: "mount /dev/sdb" → resolves to /dev/xvdb → works ✓
Debian tools see: /dev/sdb (standard)
AWS sees: /dev/xvdb (native)
```

Why Not Just Use xvdb Everywhere?

- **Portability** — standard Debian tooling, scripts, and documentation all reference `sd*`
- **Consistency** — if you ever migrate off Xen-based instances (e.g., to Nitro-based instances which use `/dev/nvme*`), only the symlink mapping changes, not every script
- **Convention** — ops teams maintain one naming standard across all infrastructure, regardless of the underlying virtualization

Quick Disk Naming Cheat Sheet

```
/dev/sda, sdb, sdc... ← traditional Linux (SCSI)
/dev/xvda, xvdb, xvd... ← AWS Xen instances
/dev/nvme0n1, nvme1n1... ← AWS Nitro instances (newer)
```

'a' = first disk (usually root/OS)

'b' = second disk (usually data – where Cassandra lives)

You [2026-06-15 20:51 UTC]

<command-message>export-session-pdf</command-message>

<command-name>/export-session-pdf</command-name> <command-args>Cassandrast</command-args>

You [2026-06-15 20:51 UTC]

Base directory for this skill: `/Users/shimpygupta/.claude/skills/export-session-pdf`

Export Session to PDF

Exports a Claude Code session (stored as a JSONL file) to a nicely formatted PDF at

`/Users/shimpygupta/Documents/Personal4/Claudesessions/`.

Session file location

Claude Code stores sessions as JSONL files under:

```
~/.claude/projects/<project-dir>/<session-id>.jsonl
```

There is typically one project directory per working directory. The project directory name is the working directory path with `/` replaced by `-` (e.g., `/Users/shimpygupta` → `-Users-shimpygupta`).

Step-by-step workflow

Step 1: Determine the session file

If the user provided a path (e.g., `/export-session-pdf ~/.claude/projects/.../abc123.jsonl`): use that path directly.

If no path was given: 1. List available sessions: ``bash ls -lt ~/.claude/projects/*/*.jsonl 2>/dev/null | head -30`` 2. Show the user a numbered list of sessions (most recent first), with the date derived from the filename timestamp. Ask which session(s) they want to export. To get a quick preview of what each session is about, read the first user message: ``bash python3 -c "import json, sys with open(sys.argv[1]) as f: for line in f: obj = json.loads(line) if obj.get('type') == 'user': msg = obj.get('message', {}) content = msg.get('content', '') text = content if isinstance(content, str) else next((b.get('text', '')) for b in content if isinstance(b,dict) and b.get('type')=='text'), '') print(text[:120]) break "`` 3. If the user says "all" or "export all sessions", export every .jsonl in the project directory.

Step 2: Run the export script

Use the bundled script at `scripts/export_to_pdf.py` (relative to this skill's directory, which is `~/.claude/skills/export-session-pdf/`):

```
python3 ~/.claude/skills/export-session-pdf/scripts/export_to_pdf.py <session.jsonl>
```

The script will print SUCCESS: `<path>` on success or an error message. The PDF is saved automatically to `/Users/shimpygupta/Documents/Personal4/Claudesessions/`.

To export multiple sessions, run the script in a loop:

```
for f in ~/.claude/projects/-Users-shimpygupta/*.jsonl; do
    python3 ~/.claude/skills/export-session-pdf/scripts/export_to_pdf.py "$f"
done
```

To export sessions from a specific date onwards, pass `--from-date YYYY-MM-DD`:

```
for f in ~/.claude/projects/-Users-shimpygupta/*.jsonl; do
    python3 ~/.claude/skills/export-session-pdf/scripts/export_to_pdf.py "$f" --from-date 2026-04-15
done
```

Sessions whose first message is before the cutoff date are silently skipped with a SKIPPED line. If the user says something like "export sessions from April 15" or "only sessions after last week", derive the `--from-date` value and include it.

Step 3: Report results

Tell the user: - The PDF filename(s) saved - How many messages were exported - If any sessions were skipped (e.g., empty or unreadable)

Notes

- The script uses `reportlab` (already installed) — no additional dependencies needed.
- Sessions with no user/assistant messages are skipped automatically.
- The PDF filename format is: `YYYY-MM-DD_<first8chars-of-session-id>.pdf`
- Tool calls are shown as `[Tool call: name(...)]`; internal thinking blocks are hidden for readability.

ARGUMENTS: `Cassandrast`

Claude [2026-06-15 20:51 UTC]

Let me find the current session and export it.

Claude [2026-06-15 20:51 UTC]

[Tool call: Bash({"command": "ls -lt ~/.claude/projects/-Users-shimpygupta/*.jsonl 2>/dev/null | head -10", "description": "List recent session files"})]

Claude [2026-06-15 20:51 UTC]

The most recent session (228KB, currently active) is this Cassandra discussion. Exporting it now.

Claude [2026-06-15 20:51 UTC]

[Tool call: Bash({"command": "python3 ~/.claude/skills/export-session-pdf/scripts/export_to_pdf.py /Users/shimpygupta/.claude/projects/-Users-shimpygupta/6d3e8798-2129-448c-aba4-77e6f2f80f24.jsonl", "description": "Ex]